

Retail RAG Assistant

Chatbot RAG song ngữ Việt/Anh cho shop thời trang: trả lời câu hỏi **sản phẩm** (giá, tồn kho, size, màu) và **chính sách** (đổi trả, vận chuyển, bảo hành, size) dựa trên dữ liệu thật của shop, hỗ trợ hội thoại nhiều lượt. Backend FastAPI + ChromaDB + Groq, frontend Streamlit, có eval pipeline đo recall/hallucination/latency — chi tiết ở phần [Kết quả](#).

Python

FastAPI

Backend

Streamlit

Frontend

ChromaDB

Vector Store

Groq

openai/gpt-oss-120b

Docker

Ready

Mục lục

Cấu trúc thư mục	Kiến trúc xử lý
Cài đặt	Chạy
Giao diện	Phương pháp đánh giá
Kết quả	Hướng phát triển

Cấu trúc thư mục

```
retail-rag-assistant/  
├── api/                                # Backend FastAPI  
│   ├── main.py                        # Khởi tạo app, lifespan, CORS, /health  
│   ├── dependencies.py                # DI: retriever, generator, condenser dùng chung  
│   (khởi tạo 1 lần lúc startup)  
│   ├── schemas.py                    # Pydantic models cho request/response  
│   └── routers/  
│       └── chat.py                    # Endpoint /chat và /chat/stream (SSE), condense +  
retrieval + generate  
├── config/  
│   ├── backend.py                    # Cấu hình dùng chung  
│   └── frontend.py                   # Đường dẫn dữ liệu, Chroma, embedding, Groq,  
condense, retrieval  
├── theme                              # URL API, văn bản UI, câu hỏi gợi ý, bảng màu  
├── data/  
│   ├── raw/styles.csv                # Dữ liệu gốc (~44.400 sản phẩm)  
│   ├── processed/  
│   │   ├── clean_styles.csv          # Dữ liệu gốc đã làm sạch  
│   │   ├── products.csv              # 5.000 sản phẩm đã chuẩn hoá song ngữ  
│   │   └── policies/*.md              # 5 file chính sách của shop  
│   └── eval/                          # Eval set (eval_set.csv + eval_set_manual.csv)  
và results.csv (build artifact)  
└── frontend/                          # Giao diện Streamlit
```

```

|   |─ app.py                # Điểm vào giao diện, vòng lặp hội thoại, gửi kèm
history
|   |─ components.py        # Header, sidebar, gợi ý nhanh, thẻ nguồn,
toolbar tin nhắn
|   |─ styles.py / styles.css # CSS tùy biến – theme gradient tím → hồng
|   |─ api.py                # Client gọi backend (đồng bộ + streaming SSE)
|
|─ notebooks/
|   |─ eda_clean_data.ipynb  # EDA dữ liệu đã làm sạch và lấy 5000 mẫu
|   |─ eda_raw_data.ipynb    # EDA dữ liệu gốc
|
|─ src/
|   |─ eval/
|   |   |─ build_eval_set.py # Sinh câu hỏi product từ chính products.csv
|   |   |─ run_eval.py       # Chạy eval, tính recall + chấm bằng Groq judge,
xuất CSV
|   |─ ingest/
|   |   |─ clean_products.py # Lọc phạm vi thời trang, sample cân bằng, dịch,
sinh giá/tồn kho/size
|   |   |─ build_index.py    # Chunk hoá sản phẩm + policy, nạp vào ChromaDB
|   |   |─ rag/
|   |   |   |─ condense.py   # Viết lại câu hỏi thành câu độc lập dựa trên
lịch sử hội thoại (multi-turn)
|   |   |   |─ generator.py  # Gọi Groq (streaming + non-streaming), ghép
history, detect ngôn ngữ
|   |   |   |─ prompt.py     # Template system prompt VI/EN – luật chống
hallucination chi tiết
|   |   |   |─ query_filter.py # Trích điều kiện giá/màu/giới tính/size từ câu
hỏi -> ChromaDB where-clause
|   |   |   |─ retriever.py   # Embedding, truy vấn Chroma, slot riêng theo
doc_type, convert distance -> similarity
|
|─ .streamlit/config.toml    # Theme màu cho các widget gốc của Streamlit
|─ docker/
|   |─ docker-compose.yml    # Chạy backend + frontend bằng một lệnh,
healthcheck + volume persist index
|   |─ Dockerfile            # Image dùng chung cho cả hai service, pre-
download embedding model lúc build
|   |─ Dockerfile.dockerignore # .dockerignore riêng cho Dockerfile này
|   |─ entrypoint.sh         # Tự build lại vector index nếu volume chưa có
sẵn ChromaDB
|─ README.md
|─ requirements.txt

```

Kiến trúc xử lý

```

data/raw/styles.csv (~44.400 dòng)
| clean_products.py – lọc về đúng phạm vi thời trang, sample cân bằng theo
articleType,
|                     dịch màu/loại/giới tính sang tiếng Việt, sinh
price/stock/size (deterministic theo id)

```

```

▼
data/processed/products.csv (5.000 sản phẩm) + data/processed/policies/*.md (5
file)
| build_index.py – 1 sản phẩm = 1 chunk song ngữ; policy chunk 150 từ, overlap
30
▼
ChromaDB (metric cosine, ~5.015 chunk)
|
| — Câu hỏi mới từ khách —————
|
| condense.py – nếu có lịch sử hội thoại, viết lại câu hỏi cuối thành câu ĐỘC
LẬP
|
| (thay đại từ, bổ sung tên sản phẩm còn thiếu) chỉ để phục vụ
retrieval,
|
| câu hiển thị cho khách và câu đưa cho Generator vẫn giữ nguyên
|
| query_filter.py – trích điều kiện giá/màu/giới tính/size từ câu hỏi (regex +
keyword)
|
| thành ChromaDB where-clause, fallback về tìm không filter
nếu rỗng
|
| retriever.py – intfloat/multilingual-e5-small, slot riêng theo doc_type (3
product + 2 policy)
▼
Groq (openai/gpt-oss-120b) — FastAPI /chat, /chat/stream (SSE) — Streamlit

```

Mô hình embedding: `intfloat/multilingual-e5-small` — 384 chiều, tối đa 512 token, huấn luyện riêng cho bài toán truy hồi bất đối xứng (query ngắn ↔ document dài). Bắt buộc thêm tiền tố **query:** cho câu hỏi và **passage:** cho tài liệu khi encode.

Độ đo khoảng cách: khai báo tường minh `cosine` khi tạo collection thay vì phụ thuộc mặc định `l2` của Chroma. Retriever đọc lại metric từ chính collection để chuyển đổi distance → similarity đúng công thức tương ứng.

Slot riêng theo doc_type: dành 3 slot cho sản phẩm và 2 slot cho chính sách thay vì gộp chung một bảng xếp hạng top-k. Lý do và số liệu trước/sau nằm ở phần [Kết quả](#).

Metadata filtering (query_filter.py): trích điều kiện cứng (giá dưới/trên X, màu, giới tính, size) từ câu hỏi bằng regex/keyword matching, ghép vào `where`-clause của ChromaDB để lọc trước khi tính similarity, thay vì chỉ dựa vào semantic search — hữu ích cho các câu hỏi có điều kiện rõ ràng ("áo khoác dưới 500k"). Là best-effort (không bắt hết mọi cách diễn đạt), nên tăng gọi (`chat.py`) luôn có fallback về tìm kiếm không filter nếu kết quả lọc rỗng.

Condense theo lịch sử hội thoại (condense.py): dùng một model Groq riêng, nhẹ hơn (`llama-3.1-8b-instant`), để viết lại câu hỏi cuối thành câu hỏi độc lập trước khi retrieval, cho phép hỏi kiểu "còn màu đen không?" sau khi đã hỏi về một sản phẩm cụ thể. Câu đã condense chỉ dùng nội bộ để retrieve — khách vẫn thấy đúng câu họ gõ.

Cài đặt

```

python -m venv venv
.\venv\Scripts\Activate.ps1      # macOS/Linux: source venv/bin/activate

```

```
pip install -r requirements.txt
cp .env.example .env # rồi điền GROQ_API_KEY
```

Chạy

```
# 1. Dựng dữ liệu (deterministic, seed 42)
python -m src.ingest.clean_products

# 2. Sinh eval set từ dữ liệu
python -m src.eval.build_eval_set

# 3. Build vector index
python -m src.ingest.build_index

# 4. Backend + frontend
uvicorn api.main:app --reload
streamlit run frontend/app.py
```

Hoặc dùng Docker cho cả hai service cùng lúc: `docker compose -f docker/docker-compose.yml up --build` (backend cổng :8000, frontend cổng :8501). `entrypoint.sh` tự động build lại vector index nếu volume `chroma_data` chưa có sẵn `chroma.sqlite3` và `docker-compose.yml` khai báo healthcheck để frontend chỉ khởi động sau khi backend sẵn sàng.

Giao diện

Chat UI dựng trên Streamlit, theme gradient tím → hồng, có gợi ý nhanh dạng chip ở sidebar, streaming câu trả lời qua `/chat/stream` (SSE, tắt được để so sánh độ trễ), thẻ "Nguồn tham khảo" dạng expander, và toolbar sao chép/đánh giá 👍 🗨️ dưới mỗi câu trả lời. Mỗi lượt hỏi gửi kèm history của phiên lên backend để hỗ trợ hỏi nối tiếp.

Phương pháp đánh giá

Nguyên tắc: câu hỏi sinh từ sản phẩm thật, **ground truth không viết tay** — luôn resolve bằng code từ `products.csv` qua cột `product_filter`.

`eval_set.csv` là build artifact, không sửa tay. Nguồn của nó gồm hai phần:

- `build_eval_set.py` sinh 75 câu (60 câu product stratified theo `articleType`, luân phiên vi/en; 15 câu `product_not_found` dùng tổ hợp thuộc tính nghe hợp lý nhưng không tồn tại trong catalog).
- `eval_set_manual.csv` giữ 30 câu policy + out_of_scope viết tay.

105 câu, chia 5 nhóm đo 5 thứ khác nhau:

Nhóm	n	Ground truth	Đo gì
<code>product/strict</code>	30	đúng 1 sản phẩm (có brand trong câu hỏi)	recall@1
<code>product/loose</code>	30	cả tập sản phẩm khớp filter	phrasing như khách thật

Nhóm	n	Ground truth	Đo gì
product_not_found	15	rỗng (tổ hợp không tồn tại)	hallucination
policy	20	tên file policy	retrieval trên corpus nhỏ
out_of_scope	10	phải từ chối	off-domain, PII, prompt injection

Chấm điểm:

- Judge **phải khác model** sinh câu trả lời — `run_eval.py` từ chối chạy nếu `GROQ_JUDGE_MODEL` trùng `GROQ_MODEL`.
- Judge prompt phân biệt strict/loose: câu loose chấm theo "có nêu được ít nhất một sản phẩm trong tập khớp", không phạt vì chọn sản phẩm khác với ví dụ liệt kê.
- Nhóm từ chối chấm bằng judge chứ không regex, tách hai chỉ số `refused` / `invented`.
- Mọi tỷ lệ in kèm **khoảng tin cậy 95% (Wilson)**, vì n mỗi nhóm chỉ 15–30.

Hai chế độ chạy không tốn quota:

```
EVAL_RECALL_ONLY=1 python -m src.eval.run_eval # chỉ đo recall, không gọi Groq
EVAL_DRY_RUN=1 python -m src.eval.run_eval # smoke test toàn pipeline
```

Mỗi chế độ ghi ra file riêng (`results_recall.csv`, `results_dryrun.csv`) để không bao giờ ghi đè kết quả thật. Mặc định `EVAL_RESUME=1`: nếu hết quota Groq giữa chừng, kết quả đã chấm được lưu lại và lần chạy sau chỉ chấm nốt phần còn thiếu.

Kết quả

Số liệu dưới đây tính trực tiếp từ `data/eval/results.csv` (105/105 câu, không có câu nào generation lỗi).

Retrieval (recall)

Nhóm	recall@1	recall@3	n
policy	100% [84–100%]	100% [84–100%]	20
product/strict	100% [89–100%]	100% [89–100%]	30
product/loose	90% [74–97%]	100% [89–100%]	30

`policy` đạt 100% nhờ giữ slot riêng theo `doc_type` (`SPLIT_BY_DOC_TYPE`) — nếu gộp chung top-k, 15 chunk `policy` dễ bị 5.000 chunk sản phẩm nhấn chìm bất cứ khi nào câu hỏi `policy` dùng từ vựng thuộc miền sản phẩm (vd "shoe size 40" khớp `Size: 40` của một SKU cụ thể).

Answer quality (Groq Judge, thang 1–5)

Nhóm	Relevance	Correctness	Faithfulness	n
policy	5.00	4.90	4.90	20

Nhóm	Relevance	Correctness	Faithfulness	n
product/strict	5.00	4.97	4.67	30
product/loose	5.00	4.47	4.33	30

- **Relevance:** mức độ trả lời đúng ý định người dùng.
- **Correctness:** mức độ chính xác của thông tin so với dữ liệu thật.
- **Faithfulness:** mức độ bám sát CONTEXT, không suy diễn ngoài dữ liệu.

Refusal / Hallucination

Nhóm	Refused	Invented	n
product_not_found	100%	0%	15
out_of_scope	100%	0%	10

Hệ thống từ chối đúng trong toàn bộ câu hỏi ngoài phạm vi và câu hỏi về tổ hợp sản phẩm không tồn tại.

Latency

Đánh giá trên toàn bộ 105 câu hỏi (bao gồm cả bước condense khi có history):

Metric	Giá trị
p50	~15.0 s
p95	~23.0 s

Phần lớn thời gian nằm ở bước sinh câu trả lời bằng Groq; retriever và ChromaDB chỉ chiếm một phần nhỏ. Độ trễ này cao hơn đáng kể so với một pipeline retrieval-only, chủ yếu do có thêm lượt gọi condense cho các câu có lịch sử hội thoại.

Kết luận

- Recall@3 **100%** trên cả ba nhóm; Recall@1 **90–100%**, thấp nhất ở **product/loose** do nhiều SKU cùng khớp filter.
- Hallucination **0%** trên cả hai nhóm cần từ chối (**product_not_found**, **out_of_scope**).
- Faithfulness **4.33–4.9/5**, Correctness thấp nhất cũng ở **product/loose** (4.47/5) — cùng nguyên nhân: nhiều SKU khớp cùng lúc, model đôi khi trình bày chưa đầy đủ thuộc tính từng SKU.

Hướng phát triển

Hai hạng mục "Structured Retrieval" (metadata filtering) và "Multi-turn Conversation" từng nằm ở phần này đã được triển khai (**query_filter.py**, **condense.py**). Các hướng còn lại đáng cân nhắc tiếp:

- **Giảm độ trễ:** p50 hiện ~15s, phần lớn do 2 lượt gọi LLM tuần tự (condense rồi generate) — có thể chạy condense sớm hơn/song song, cache câu hỏi lặp, hoặc bỏ condense khi câu hỏi rõ ràng không phụ thuộc ngữ cảnh.
- **Mở rộng query_filter:** từ điển màu/giới tính hiện chỉ phủ tiếng Việt phổ biến nhất mỗi màu và cú pháp giá kiểu Việt ("dưới 500k") — bổ sung cú pháp tiếng Anh, khoảng giá (từ...đến), và nhiều biến thể diễn

đạt hơn.

- **Feedback Learning:** nút 👍 / 🗨️ trong UI hiện chỉ lưu trong session, chưa được ghi lại và phân tích lâu dài để cải thiện prompt/retrieval theo thời gian.
- **Mở rộng dữ liệu:** bổ sung dữ liệu sản phẩm thực tế (giá, tồn kho, hình ảnh, đánh giá).